

General Audit

Pre-production review of the web apps, backend, and mobile app — every finding verified twice.

20 July 2026

68

CONFIRMED
FINDINGS

2

HARD PROD
BLOCKERS

3

REVIEW
TEAMS

0

FALSE
ALARMS

Executive summary

This audit was run because the platform is going to production soon. Three independent review teams read the actual source code — not commit messages — across the three codebases. Every candidate finding was then handed to a second, skeptical reviewer whose job was to *disprove* it; only findings that survived that challenge are in this report. Nothing here is speculation.

Team	Findings	What it covered · headline
1 — Platform Web apps + Backend	36	Client panel, admin panel, shared components, and the Django backend (branch staging, the code that goes to prod). Headline: tenant-isolation holes in the labels/ annotations feature, and the admin app can't reach the backend at all.
2 — Mobile adaptation What mobile must change	15	How the Flutter app must change to match the backend + web changes (work-tasks → activities, condition-at-start). Headline: mobile is a whole migration behind and would break against the new backend.
3 — Mobile audit General mobile bugs	17	A thorough bug hunt on the Flutter app itself (Bluetooth, recording, sync, data). Headline: silent data loss and corruption in what gets recorded and uploaded.

The two hard blockers — fix before any prod release

C1 · The labels & annotations feature is not walled off between companies. In one backend file the security checks that keep each organization's data private were switched off (commented out). Right now any logged-in user could read — and even write into — another company's session labels and categories. For a product sold to multiple companies, this is the most serious kind of issue.

C2 · The admin panel points at the wrong address. The internal admin app is hard-wired to talk to "localhost" (the viewer's own computer) instead of the real server. Deployed to production, the admin panel simply won't work — every action fails after login.

Two patterns showed up across all three codebases

A half-finished rename. Session statuses were renamed in the database ("ongoing" became "recording", "finished" became "completed", etc.), but the change wasn't carried through everywhere. It now breaks a backend auto-archiving rule *and* the mobile app's live status reporting.

A half-finished migration. "Work tasks" are being replaced by "activities" on a new backend module. The backend built it and the web app already switched over — but the **mobile app hasn't started**, and the condition-at-start feature isn't even on the mobile dev branch yet. That gap is Team 2's entire report.

How to read this report

Every item below starts with an **“In plain terms”** box — a jargon-free explanation of what the thing is, where it lives, and why it's a problem. Under that is the **technical detail** (exact file, line, and failure path) for the developer who will fix it. You can read only the blue boxes and still understand the whole report.

■ **Critical / Blocker** — do not ship with this open ■ **High** — wrong data, crashes, or security
■ **Medium** — real bug, contained impact ■ **Low** — minor / cosmetic / latent

Note on counts: teams reported 68 raw findings; a few were the same bug found by two reviewers. This report presents each distinct bug once, so the numbering per section may total slightly less than the raw count. Nothing was dropped.

Backend audited on branch staging (the branch that deploys to production; the dev branch is 631 commits behind). Web apps audited on branch dev.

Critical

A1 **CRITICAL** Other companies' session labels are readable and writable by anyone

backend · work_tags/views.py:152 & 164 (EventListCreateAPIView)

In plain terms: The platform serves many companies, and each company's data is supposed to be private to them. This endpoint hands back the "events / labels" of a work session (the tags an ergonomist places on a recording) based only on the session's ID in the web address. The security check that confirms "this user actually belongs to that company" has been commented out. So any logged-in user, from any company, can pull up another company's session annotations — and can even POST fake events onto someone else's session. This is a live cross-company data leak **and** data-tampering hole.

Technical: `permission_classes = [IsAuthenticated]` with the `IsUserInSessionOrganization` guard commented out; `get_queryset` filters solely by the `session_id` URL kwarg. The neighbouring `EventBulkCreateAPIView` still enforces `SessionCrudPermission`, proving the check was intended here and was removed. Fix: restore the org-scoping permission and validate session ownership on `perform_create`.

A2 **CRITICAL** The admin panel is wired to "localhost" and won't work in production

web · apps/wergonic-admin/src/services/axios.ts:7

In plain terms: The internal admin app needs to know the web address of the backend server to talk to it. Instead of reading that address from configuration, someone hard-coded it to `http://localhost:8000` — which means "this same computer." When the admin app is deployed and a super-admin opens it, every request goes to their own machine, not the Wergonic server, so nothing loads. The app is dead on arrival in production, even though the correct configuration values already exist and are injected at build time.

Technical: `axios.create({ baseURL: "http://localhost:8000/api/v1/" })` ignores `VITE_API_BASE_URL` / `VITE_API_VERSION` (both defined in the Dockerfile and `.env.example`). The sibling client-panel does it correctly: `baseURL: `${import.meta.env.VITE_API_BASE_URL}${import.meta.env.VITE_API_VERSION}``. One-line fix.

A3 **CRITICAL** The whole "categories & labels" module trusts the company ID from the web address

backend · work_tags/views.py:45, 108, 130, 299 (Category / NestedCategory / Label views)

In plain terms: Same root cause as A1, spread across the rest of the labels feature. The label *categories*, the label *lists*, and the "create a label" action all decide which company's data to touch based purely on a company ID sitting in the URL — with no check that the requester belongs to that company (the guards are commented out). So by changing the ID in the address bar, a user of Company A can list, create, edit, or delete Company B's label categories and labels, and even inject labels into Company B's categories. It's the same door left open in several rooms.

Technical: `CategoryListCreateAPIView` / `CategoryDetailAPIView` (l.45), `LabelListCreateAPIView.perform_create` (l.108), `LabelListView` (l.130), and `NestedCategoryViewSet` (l.299) all resolve org from `self.kwargs["org_id"] / category_id` with `permission_classes = [IsAuthenticated]` only; `IsOrgAdminOrErgonomist / IsUserInCategoryOrganization` commented out. Fix as a set alongside A1 — restore permissions + scope querysets to the requester's org, never the URL's.

High — security & access

A4 **HIGH** Support tickets are readable and deletable by any user

backend · tickets/views.py:170

In plain terms: The support-ticket screen returns every ticket on the platform to any logged-in user, and lets them delete any ticket by its number. Tickets contain other users' email addresses and message contents. So one user can read everyone's support conversations and delete other people's tickets.

Technical: `TicketRetrieveDestroyView` and `TicketListCreateView` use `queryset = Ticket.objects.all()` with only `IsAuthenticated` and no sender/org scoping (classic IDOR). Fix: filter by `sender=request.user` (or org) and block cross-user delete.

A5 **HIGH** Another company's AI report can be read directly

backend · user_sessions/views/stats_view.py:1118

In plain terms: There's an AI-generated analysis attached to each session. The "save" side of this endpoint correctly checks the company, but the "read" side doesn't check at all — it just looks the session up by ID and returns its AI report. So anyone can fetch another company's AI ergonomic analysis by guessing/knowing a session ID.

Technical: `GenerateGeminiAnalysisView` (the routed one) is `IsAuthenticated` only; its `get()` does `get_object_or_404(Session, id=pk)` then returns `session.ai_report` with no org check. A secured variant (`IsInSessionOrganization`) exists in `stats/gemini.py` but is never wired into urls. Fix: route the secured variant, or add the org check to `get()`.

A6 HIGH A session can be secretly re-pointed to another company's activity/condition

backend · work_library/views.py:265 (SessionActivity / SessionCondition detail)

In plain terms: This is in the new "activities/conditions" feature. When you first attach an activity to a session, the backend checks the activity belongs to your company. But when you *edit* that attachment afterward, it forgets to re-check. So a user can edit their own session's row to point at another company's activity — which leaks that activity's name and inflates the other company's usage counters with your session.

Technical: `SessionActivityDetailView / SessionConditionDetailView` scope the row by `session__organization` but have no `perform_update`; the serializer's `activity` field is a writable `PrimaryKeyRelatedField` with an all-org queryset. The create path guards this (`activity.group.organization_id != session.organization_id`) — the update path must too.

A7 HIGH Tasks without a linked model are exposed to every company

backend · workcycles/views.py:173

In plain terms: The work-cycle "tasks" list is meant to show only your own company's tasks. But it also includes any task that isn't linked to a parent model (which can happen with legacy or imported data), and those unlinked tasks are shown to — and editable/deletable by — everyone, regardless of company.

Technical: `TaskViewSet.get_queryset` ORs in `Q(task_model__isnull=True)`, so orphan tasks bypass the per-org restriction on list/retrieve/update/destroy. Fix: drop the null branch or scope it.

High — wrong calculations (assessment accuracy)

A8 HIGH RAMP over-scores neck "backward bending" risk

backend · ramp/calculations/ramp_calculator.py:278

In plain terms: RAMP is one of the ergonomic risk scores. One rule is supposed to count time the worker's head is bent backward more than 10 degrees. Because of a wrong threshold, it actually counts anything past 5 degrees as if it were past 10. That makes the neck risk score come out higher than it should — the tool reports more risk than really exists, and disagrees with every other calculation pipeline that uses the correct 10-degree line.

Technical: `calculate_time(head, field="forward_bending", upper_threshold=-5)` feeds a field named `...backward_bending_gt_10_degrees...`; the matcher returns `val <= -5`. Canonical criteria (`posture_thresholds.head_le_neg10`) use -10° . Fix: threshold -10 , not -5 .

A9 HIGH MEC arm-position score ignores one arm's standing-still risk

web · apps/client-panel/src/features/MEC/utils/calculations.ts:297

In plain terms: For the MEC arm-position question, the app looks at both arms but then throws one away — it keeps only the arm that moved more often, and scores just that one. The problem: risk can also come from an arm being held *static* for too long. So if the left arm barely moved but was held still past the danger line, that arm should be RED — but the app picks the busier right arm and reports YELLOW. The screen even shows both arms' numbers next to a status that contradicts them.

Technical: `getMECQuestionCalculations([rightArmFreq >= leftArmFreq ? right : left])` — the discarded arm's static-minutes threshold (an independent RED/YELLOW trigger) is never evaluated. Fix: evaluate both arms and take the worse status.

High — session merge is broken

A10 HIGH Merging two sessions crashes and marks the result "failed"

backend · user_sessions/views/session_views.py:626

In plain terms: When you merge two recordings of the same worker into one, both recordings contain the same body parts (e.g. both have a "left arm"). The merge just moves all the body-part records onto the new combined session, so now there are two "left arm" records. Later, when the system re-processes the merged data, it expects exactly one record per body part, finds two, throws an error, retries, and finally gives up and marks the merged session **failed**. Because both sources always share body-part names, essentially every real merge breaks.

Technical: `session.measurement_set.update(session=merged_session)` for each source produces duplicate `Measurement(session, name)` rows; reprocessing's `get_or_create(session, name)` raises `MultipleObjectsReturned`. No unique constraint on `(session, name)`. Fix: de-duplicate/rename limbs on merge, or add a merge-aware processing path.

A11 HIGH Merged sessions get their movement axes swapped, corrupting the results

backend · user_sessions/utils/measurements.py:143

In plain terms: Recordings carry a small metadata file that says which app version made them. The app uses that to decide whether an old "axis-swap" correction is needed (older recordings need it, newer ones must not get it). When sessions are merged, the merge forgets to include that metadata file. With it missing, the system assumes a very old version and applies the swap to everything — so merged results from modern recordings come out with their "forward" and "side" bending swapped. The numbers are simply wrong.

Technical: `merge_measurement_files` writes only the merged CSVs into the new zip, no `.json`. Reprocessing then defaults `app_version="2.9.9"` → `use_swapping=True`, running `preprocess_dataframe_with_swapping` on sessions recorded with app > 2.9.10. Fix: re-emit the metadata json in the merged zip.

High — reliability & front-end

A12 HIGH A background job runs far too often, in every server process at once

backend · core/settings.py:334 (scheduler + cron)

In plain terms: A housekeeping job that checks session status is meant to run every ~5 hours. Because of two overlapping schedulers, it's actually scheduled to run **every single minute**, and separately it also starts up inside every server worker process. The result is the same heavy job firing many times over, hammering the cloud database (Firestore) with redundant reads and racing against itself. In production that means unnecessary cost and unpredictable behaviour.

Technical: `CRONJOBS` registers `session_status_check` at `* * * * *`; `scheduler/apps.py ready()` also starts a per-process `BackgroundScheduler`. The in-process `threading.Lock` only guards one process. Fix: single scheduler (one process / management-command lock), correct the cron cadence.

A13 HIGH One failed request can freeze the whole app on a loading spinner

web · apps/client-panel/src/hooks/auth.ts:26

In plain terms: When a returning user opens the client panel, the app fetches their profile before showing anything. If that one request hiccups (a momentary network blip or the server waking up), the code has no safety net — it throws, never flips the "done loading" switch, and the app sits on a full-screen spinner forever. The only escape is repeatedly reloading until a request happens to succeed. The admin app already handles this correctly; the client panel doesn't.

Technical: `onAuthStateChanged` awaits `retrieveCurrentUser()` and reads `.data.user.role` with no try/catch; a rejection or missing `user` skips `setPendingAuthState(false)`, so `pendingAuthStateAtom` stays true → permanent `<GlobalLoader/>`. Fix: wrap in try/catch, null-guard, and always clear the pending flag.

A14 HIGH Changing "rows per page" on any table breaks it with an error

web · packages/ui/src/components/Tables/PaginatedTableV2/PaginatedTableV2.tsx:232

In plain terms: On any list (Users, Devices, Organizations, Workers...), if you're on, say, page 3 and then increase how many rows show per page, there may no longer be a page 3. The table forgets to jump back to page 1, so it asks the server for a page that doesn't exist, gets an error back, and shows an empty/broken table until you fix the URL by hand. This shared table is used all over both apps, so it affects many screens.

Technical: the page-size `onChange` sets `page_size` but never resets `page` to 1 (unlike the search/filter inputs, which call `resetURLPageParam`); server-side pagination then returns DRF 404. Fix: reset `page=1` on page-size change.

Medium

A15 MEDIUM A MEC assessment can silently finish with missing data

backend · ramp/calculations/mec_segments.py:277

In plain terms: In a specific data situation (the trunk sensor has side-bending readings but no forward-bending readings), the trunk part of the MEC calculation hits an internal error. That error is quietly caught and hidden, so the assessment is saved as "completed" but is actually missing several trunk/head results and its "missing body part" flags — with no warning to the user. They trust a result that's incomplete.

Technical: `trunk_time_array` is only assigned inside `if forward_bending_available:` but used inside `if side_bending_available:` → `UnboundLocalError`, swallowed by the broad `except Exception` at l.379, skipping sections 10.2/11 and the missing-limb block. Fix: define `trunk_time_array` unconditionally.

A16 MEDIUM Per-task RAMP mixes in whole-session hand-distance data

backend · ramp/calculations/ramp_calculator.py:435

In plain terms: RAMP can be run for a single task inside a session. Five of its six posture measures correctly look only at that task's time window — but the sixth (how far the hands are from the body) mistakenly uses the whole session instead. So a task-specific report is contaminated by hand-distance behaviour from the rest of the session.

Technical: postures 1–5 use `get_filtered_measurements(task_id, start_time, end_time)`; posture 6 reads `SessionStats.get_zone_counts()` for the whole session with no task filter. Fix: scope zone counts to the task window.

A17 MEDIUM Excel export writes head data on top of trunk data

backend · user_sessions/views/stats_view.py:1022

In plain terms: When a single session's stats are exported to Excel, the head's forward/side movement is written into the same spreadsheet cells as the trunk's — so the head values overwrite the trunk values. An ergonomist opening the file reads head numbers where they think they're reading trunk numbers.

Technical: both trunk and head `get_values_and_durations(...)` target row 45 (cols D/F and J/L); head runs last and overwrites. The multi-session export uses distinct rows (50 vs 66), confirming a copy-paste bug. It also reuses the trunk band dictionary for head.

A18 MEDIUM A per-minute job crashes on sessions with a deleted worker

backend · user_sessions/management/commands/session_check.py:38

In plain terms: An auto-finish job (which runs every minute) looks up a session's company by walking session → worker → user → organization. But a worker can be deleted (leaving that link empty). When it hits such a session it crashes, and because there's no error handling around the loop, every other expired session waiting behind it is skipped for that minute.

Technical: `.document(str(session.worker.user.organization.id))` — `worker` is `SET_NULL`, `User.organization` is nullable → `AttributeError` aborts `handle()`. Also semantically wrong: should use `session.organization_id`. Fix: use the session's own org and guard nulls.

A19 MEDIUM The "auto-archive ongoing sessions" rule archives nothing

backend · user_sessions/management/commands/archive_sessions.py:37

In plain terms: This is the half-finished rename showing up. Users can set a rule to auto-archive "ongoing" sessions. But sessions no longer store the word "ongoing" — after a database rename they store "recording." The rule still searches for "ongoing," matches zero sessions, and quietly does nothing. The feature looks configured but never fires, with no error.

Technical: migration 0096 renamed `Session.Status ONGOING = "RECORDING"`, but `ArchiveSetting.Status` still offers only `ONGOING/PAUSED/FAILED`, and the cron filters `status=rule.condition_value`. Also, one malformed rule aborts archiving for all users (no per-rule try/except). Fix: align the allowed values with the renamed statuses.

A20 MEDIUM "Your measurement failed" alerts can go to the wrong place or nowhere

backend · notifications/models.py:50

In plain terms: When a session fails to process, the user should get a notification. But the notification is filed under whichever company the user is *currently* looking at, not the company the failed session actually belongs to. If the user has no active company at that moment, the alert is never delivered — they never learn their measurement failed. If they've switched companies, the alert is filed under the wrong one.

Technical: the Firestore push signal resolves org via `recipient.get_active_consulting_organization()` at save time. If it returns `None`, the doc is never written and `organization` stays null; `NotificationListView` filters strictly by active org, so it never shows. Fix: tie the notification to the session's org.

A21 MEDIUM Archived sessions keep nagging in the Home to-do inbox

web · apps/client-panel/src/features/home/hooks/useHomeInbox.ts:30

In plain terms: The Home page shows to-do lists like "sessions that failed" and "sessions ready for an assessment." These lists don't exclude archived sessions, and the backend returns archived ones by default. So after you archive a session to get it out of your way, it keeps appearing in the to-do inbox and inflating the counts — and you can't clear it because it's already archived.

Technical: both `useHomeInbox` queries call `listOrganizationSessions` without `is_archived=false`; `StartSession.get_queryset` has no archived filter. Fix: pass `is_archived=false` (see also A29).

A22 MEDIUM Multi-task MEC results show duplicate, unlabeled rows

web · apps/client-panel/src/features/assessmentsV2/components/MECResultsV2/MECResultsV2.tsx:108

In plain terms: When a MEC assessment covers several work tasks, the detailed results table repeats each sub-question once per task but never prints the task's name next to it. So you see two rows both labeled "1.1" with different numbers and no way to tell which task each belongs to. On top of that, each row's colour badge shows the overall total's risk level instead of that task's own level.

Technical: rows built via `flatMap` carry a `task_label` that the table never renders, and are keyed by `sub.question` (colliding React keys). The Zone badge reads `item.status` from the TOTAL row. Fix: render `task_label`, key uniquely, use per-task status.

A23 MEDIUM Add / edit organization: a save error throws away everything you typed

web · admin · AddOrganization.hooks.ts:68 & EditOrganizationForm.hooks.ts:28

In plain terms: In the admin panel, when you fill out the "new organization" form (or edit one) and the save fails — say a duplicate email or a network blip — the form modal closes anyway and discards everything you entered. You see an error, but the modal is already gone and you must re-type the whole form. It closes on failure exactly as if it had succeeded.

Technical: the mutation closes the modal in `onSettled`, which runs on both success and error. Fix: close only in `onSuccess`.

A24 MEDIUM The "Run assessment" form wipes your choices a moment after opening

web · apps/client-panel/src/features/measurements/components/RunAssessmentModal/
RunAssessmentModal.tsx:92

In plain terms: When you open "Run assessment," the session's duration loads a fraction of a second later in the background. When it arrives, a reset routine re-runs and clears the form — so if you were quick and already picked your method and settings, they silently vanish and you have to start over.

Technical: `useEffect([open, durationSeconds])` calls `setValues(assessmentDefaults(durationSeconds))`; `durationSeconds` changes 0 → real value after fetch, re-firing the reset. Fix: initialize once, or reset only on `open`.

A25 MEDIUM Admin filters break on names with "&" or spaces

web · apps/wergonic-admin/src/api/apiRoutes.ts:18

In plain terms: When you filter admin lists by, for example, an organization named "Johnson & Johnson," the special characters aren't encoded for the web address, so the server misreads the request and returns the wrong results (or none). Any filter value with an ampersand, plus sign, hash, or spaces is affected.

Technical: query builders concatenate raw values (`organization_names=${...}`, `search=`, `country=`) with no `encodeURIComponent`. Fix: URL-encode all interpolated params.

A26 MEDIUM The organization filter silently drops your earlier selections

web · apps/wergonic-admin/src/components/URLOrganizationFilter/
URLOrganizationFilter.hooks.ts:49

In plain terms: If you've already filtered by organization "A," then reopen the filter, search for "B," and add it — closing the popover replaces "A" with just "B" instead of keeping both. The filter never remembers what was already applied, so selections you didn't touch get quietly removed.

Technical: `formattedData` is rebuilt from search results with `checked: false` and never seeded from the existing `organization_names` URL param; on close it overwrites from only the loaded checkboxes. Fix: seed from current URL state.

Low

A27 LOW Home "failed/interrupted" count includes archived sessions

web · apps/client-panel/src/features/home/hooks/useHomeSummary.ts:33

In plain terms: The small summary card on Home that counts failed/interrupted sessions also counts archived ones, so the number looks higher than the real, active count. (Same root cause as A21.)

Technical: `failedInterruptedQuery` omits `is_archived=false`; count includes archived rows. Fix with A21.

A28 LOW After adding a user, the org counts don't refresh

web · apps/wergonic-admin/.../AddNewUser/AddNewUser.hooks.ts:122

In plain terms: When an admin adds a user to an organization, the code tries to refresh the affected lists — but it refreshes a name that doesn't match any real list, so nothing updates. The worker/admin counts keep showing the old numbers until a full manual page refresh.

Technical: `invalidateQueries({ queryKey: ["organizations","users"] })` matches neither `["organizations", {params}]` nor `["users", {params}]`. Fix: invalidate the correct keys.

B · Mobile — Adaptation to backend & web changes

Team 2 · 15 findings

Read this first — the big picture for mobile

The backend built a brand-new "work-library" module (activities & conditions replacing work-tasks & labels) and the web app already switched to it. **The mobile app has not.** On the mobile dev branch, the app still calls the old endpoints, still sends the old status words, and the whole "declare a condition at the start" feature **isn't there at all** — it exists only on a separate branch (migration_to_native). The items below are everything mobile must change so it doesn't break the moment the backend retires the old endpoints. "Blocker" here means: uploads fail or data is stored wrong in production.

Blockers — mobile breaks against the migrated backend

B1 **BLOCKER** Mobile still calls the old endpoints; the backend moved to a new module

flutter · lib/core/constants/api_endpoints.dart:38–45

In plain terms: Think of the backend as a building. The "labels/activities" department moved to a new address (/work-library/...), and the web app already updated its address book. The mobile app is still mailing everything to the old address (/categories/...). It works today only because the old address hasn't been shut down yet. The moment the backend closes it, every activities/labels action on mobile fails with "not found."

Technical: mobile targets legacy /categories/*; backend mounts a new app at /api/v1/work-library/* (activity-groups, activities, condition-groups, conditions, per-session activities/conditions). Fix: add /work-library/* datasources and retire the /categories/* ones.

B2 **BLOCKER** Mobile reports session status with the old words

flutter · lib/features/session/data/models/work_session_status/
work_session_status.model.dart:22

In plain terms: The database renamed session statuses — "ongoing" became "recording," "finished" became "completed." Mobile still writes the old words. Since the live status dashboard on the web now looks for the new words, it no longer recognizes a mobile recording as active, and disconnect detection breaks. If mobile ever sends the old word on a save, the backend now rejects it outright.

Technical: toMap serializes status.name.toUpperCase() → "ONGOING"/"FINISHED"; backend (migration 0096) expects RECORDING/COMPLETED/DISCONNECTED and SessionUpdateSerializer.status is a restricted ChoiceField (400s on old values). Fix: explicit value map, add a DISCONNECTED state.

B3 **BLOCKER** Choosing an operator returns every work-cycle, not that operator's

flutter · lib/features/work_cycle/data/sources/remote/
work_cycle.remote.datasource.dart:130-131

In plain terms: When a user picks an operator to see that operator's work-cycles, mobile asks the backend using the wrong label for the filter (it says "workstation" where it should say "operator"). The backend doesn't recognize the label, ignores it, and hands back *all* work-cycles in the company. So the operator drop-down effectively does nothing — you always see everything.

Technical: sends `{'workstation': operatorId}`; backend `TaskModelViewSet` only has `filterset_fields = ["operator"]`. Unknown params are silently ignored by DjangoFilter. Fix: change the key to `operator`.

B4 **BLOCKER** A task with no time value crashes the whole task list

flutter · lib/features/work_cycle/data/models/work_cycle_task/work_cycle_task.model.dart:11,22

In plain terms: Mobile assumes every task always has a duration number. But the backend allows a task to have no duration (for example, tasks imported from Excel where the time column was left blank). When mobile meets one of those, it can't handle the empty value and crashes the entire task list for that work-cycle — meaning the operator can't even start recording.

Technical: `required double duration` with `duration: json['duration'];` backend `Task.duration` is `FloatField(null=True)`. Fix: `double?` or `(json['duration'] as num)?.toDouble() ?? 0`; relax the local DB column too.

B5 **BLOCKER** Work-cycle timing is never saved because of a spelling mismatch

flutter · lib/features/settings/data/models/settings.model.g.dart:104,141

In plain terms: Mobile tags a recording's type as "WorkCycle" (no space), but the backend only recognizes the exact phrase "Work Cycle" (with a space). Because the phrase doesn't match, the backend parses zero work-cycle events — so the per-step timing of a work-cycle session is silently thrown away and never stored.

Technical: the JSON enum map emits `'WorkCycle' / 'workTask'`; backend `build_task_specs_from_metadata` matches literally `"Work Cycle" / "Work Task"`. Fix: annotate with `@JsonValue('Work Cycle') / @JsonValue('Work Task')` and regenerate.

B6 **BLOCKER** "Work break" markers are sent with an ID the backend doesn't have

flutter · lib/features/work_labels/domain/entities/work_task.entity.dart:12-16

In plain terms: Every time a worker takes a break, mobile records that using a hard-coded label ID of "0," assuming the backend has a matching label numbered 0. Backend IDs actually start at 1, so there's no label 0. The break upload is therefore rejected, and every work-break marker fails to save (and can break the whole session save).

Technical: `WorkTaskEntity.workBreak()` pins `id:0`; the bulk-create serializer validates label IDs as existing PKs. Fix: resolve the real Work-Break label ID at runtime, or have backend seed a canonical one and confirm it on staging/prod.

B7 **BLOCKER** Nested label categories crash the labels screen

flutter · lib/features/work_labels/data/models/work_category/work_category.model.dart:15,33

In plain terms: Categories can have a parent category. The backend sends the parent as a number (its ID), but mobile expects the parent to be text. As soon as any category actually has a parent, mobile can't read it and the entire labels screen fails to load. It only works today because the categories in use happen to have no parent.

Technical: `parent: json['parent'] as String?` but backend serializes the `TreeForeignKey` as an int PK. Fix: change `parent` to `int?` and parse accordingly.

B8 **BLOCKER** Body-part names are sent in a format the backend rejects

flutter · lib/features/bluetooth/.../sensor_connection_cubit.dart:143

In plain terms: Mobile sends body-part names in display form — "Right Arm," "Left Arm" — but the backend only accepts a compact internal form — "rightArm," "leftArm." Because none of the display names match, the backend concludes the session recorded *no* body parts and marks all of them missing. The web's limb filter then finds nothing for that session.

Technical: limb built from `Limbs.toString()`; backend keeps only values in `{rightArm, leftArm, forearm, hand, trunk, head}` for `recorded_limbs`. Fix: serialize with `toCamelCaseString()`. (Note: `chestStrap` has no canonical mapping — needs a decision.)

B9 BLOCKER Right-handed workers are recorded as left-handed

flutter · lib/features/session/data/models/work_session/work_session_model.dart:73-103

In plain terms: Mobile sends the worker's dominant arm capitalized ("Right"), but the backend's logic checks for lowercase ("right"). Because "Right" ≠ "right," the check fails and the worker is treated as left-dominant. That flips which arm the risk calculations focus on, corrupting the derived results for every right-handed worker.

Technical: `dominant_arm` from `DominantArm.toString()` → "Right"/"Left"; `transformations.py:362` compares `== 'right'`. Fix: send lowercase "right"/"left".

High

B10 HIGH The condition-at-start feature isn't on the mobile dev branch yet

flutter · exists only on origin/migration_to_native (not dev)

In plain terms: The whole "declare a condition at the start of a session" capability — which the backend and web already support — simply doesn't exist on the mobile branch we're preparing (dev). It was built on a different branch that also reorganized a lot of the code. So before any of the other mobile fixes even apply, that work has to be brought over onto dev. Without it, mobile users can't declare conditions and the web shows nothing for mobile sessions.

Technical: the metadata field, local DB column, config-flow step, and cubit setter live on `origin/migration_to_native` (WH-502/530/537/546), which also moved `lib/features/session/*` → `lib/features/sessions/session/*`. Fix: port that work onto `dev` first, then apply B1-B9.

B11 HIGH Mobile ignores the task order set by the ergonomist

flutter · lib/features/work_cycle/data/models/work_cycle_task/work_cycle_task_model.dart:9-24

In plain terms: On the web, an ergonomist arranges the steps of a work-cycle in a specific order (drag-and-drop). Mobile doesn't read that order field at all, so it presents the steps in whatever order the server happened to return them. The operator can end up stepping through the cycle in the wrong sequence.

Technical: model has no `order` field; backend added `Task.order` and a reorder endpoint, and web sorts by `order ?? 0` then id. Fix: read `order` and sort by it.

B12 HIGH Mobile reads a label "type" field the backend removed

flutter · lib/features/work_labels/data/models/work_label/work_label_model.dart:13,21

In plain terms: The concept of a label's "type" moved — it used to live on the label, now it lives on the category. Mobile still reads it from the old spot on the label, where it's now always empty. So any behaviour that depended on knowing whether something is a "work task" vs a "work label" is quietly dead on mobile.

Technical: `Label.type` was dropped (migration 0009); the distinction moved to `Category.category_type` (WORK_TASK/WORK_LABEL). Fix: read `category_type` from the category; deprecate `WorkLabelModel.type`.

B13 HIGH A stray character corrupts device info in lite mode

flutter · lib/features/session/presentation/pages/lite_mode_calibration.page.dart:205

In plain terms: In "lite mode," the string that packs together a sensor's body-part, serial number, and firmware version has an extra stray "}" character in it. When the backend splits that string apart, the junk character lands in one of the fields, so that device's body-part or firmware is stored wrong (and that body part gets excluded from the session).

Technical: the built string `'${selectedLimb.toString()}:...'` includes a trailing `}`; the backend splits on `:` into `{limb,device_serial,firmware_version}`. Fix: remove the stray brace and use `toCamelCaseString()` (with B8).

Medium

B14 MEDIUM Guardrail: do NOT rename these "work_task" fields

flutter · lib/features/work_cycle/data/sources/remote/work_cycle.remote.datasource.dart:157

In plain terms: This one is a warning, not a bug. While doing the "work-tasks → activities" rename, it would be tempting to rename *every* mention of "work_task." But some of them belong to a different, unrelated feature (work-cycle task filtering and assessment fields) that the backend and web intentionally kept unchanged. Renaming those would break things that currently work. Whoever does the sweep should leave them alone.

Technical: `/workcycles/tasks/?work_task=<id>` is a factory-task filter, and the assessment `work_task*` fields / `label_task|work_cycle_task` enum are unchanged on backend+web. Only `work_tags` (categories/labels) → `work_library` (activities/conditions) vocabulary changes.

B15 MEDIUM Mobile can only create one type of label category

flutter · lib/features/work_labels/data/sources/remote/
work_categories.remote.datasource.dart:60

In plain terms: When creating a category from mobile, the app always tells the backend it's a "work task" category, ignoring what the user actually selected. So users can never create the other kind ("work label") category from mobile — the feature is half-implemented, and it fails silently (the request succeeds, just always as the wrong type).

Technical: `createWorkCategory` hardcodes `'category_type': 'WORK_TASK'`; backend accepts `WORK_TASK` or `WORK_LABEL`. Fix: pass the user's selected type.

A thorough bug hunt on the Flutter app itself (branch dev): Bluetooth, recording, sync, and data handling. These are independent of the migration in Section B.

High

C1 **HIGH** GPS/location recording dies after the very first session

```
flutter · lib/features/session/domain/usecases/data_processing/  
get_location_data.usecase.dart:19
```

In plain terms: If location tracking is on, it works for the first session of the app run and then never works again. The internal "pipe" that carries location data is closed when the first session ends and is never re-opened, so every session after that records zero location data — silently. The user has no idea their later sessions are missing all GPS/activity tracking.

Technical: a `final` single-subscription `StreamController` is `close()`d in `stopStream()` and never recreated; the usecase is an app-level singleton, so session 2 reuses the closed controller (and can throw "already listened"). Fix: recreate the controller per session (or use a broadcast controller).

C2 **HIGH** Hand recordings can be uploaded with their data columns shifted

```
flutter · lib/features/measurements/domain/entities/processed_data.entity.dart:102
```

In plain terms: The uploaded data file for a hand is a table with a fixed set of column headers. But when a certain setting (vibration detection) is turned off, the app stops writing several of those columns in the data rows — without removing them from the header. Everything after that point shifts left, so values line up under the wrong headers. The backend then reads corrupt hand data for the entire recording.

Technical: the CSV header always includes the 8 vibration columns + the ref-gyro column, but each row emits them only when non-null (vibration off → empty), shifting every subsequent value. Fix: emit placeholders for the missing columns so headers and rows stay aligned.

C3 HIGH "Finish session" can report success even when it failed to save

flutter · lib/features/session/.../work_session_cubit.dart:316

In plain terms: When a user taps "Finish," if the final save step fails, the app shows an error *and* still navigates them to the summary screen as if it worked. So the user believes the session was saved and completed, when in fact it wasn't fully finalized and the upload is incomplete. It sends mixed signals: an error popup and a "success" screen at the same time.

Technical: `updateWorkSessionStatus()` swallows its error (emits `sessionError`, returns normally); `terminateSession` then unconditionally emits `sessionTerminated`, which navigates to the summary. Fix: only navigate on genuine success.

C4 HIGH If Bluetooth is off, the scan screen spins forever with no message

flutter · lib/features/bluetooth/data/repositories/ble.data.repository.dart:112

In plain terms: If a user opens the sensor-scan screen while Bluetooth is turned off, the app should tell them "Bluetooth is off." Instead, the error gets lost internally and never reaches the screen, so the user is left staring at a spinning "scanning..." indicator indefinitely, with no explanation and no way forward.

Technical: the scan subscription has no `onError`; the datasource's `addError(BluetoothException)` becomes an uncaught async error that never reaches the cubit, which stays in `scanning()`. Fix: add an `onError` path that surfaces a "Bluetooth off" state.

C5 HIGH Female workers are silently recorded as gender "other"

flutter · lib/core/enums.dart:55

In plain terms: Because of a small case-sensitivity slip, the app stores gender in uppercase ("FEMALE") but reads it back checking for lowercase ("female"). They never match, so any female worker is quietly interpreted as "other." No crash — just wrong data feeding the ergonomic reports.

Technical: `Gender.fromString` lowercases only the male branch; the female branch compares raw `== "female"` against a stored "FEMALE". Fix: lowercase both comparisons.

C6 HIGH A malformed profile can crash the work-cycle list

flutter · lib/features/work_cycle/data/models/work_cycle_model/work_cycle_model.model.dart:21

In plain terms: When reading a work-cycle record, the app assumes several nested pieces (operator → workstation → line → factory) are always present. If any one of them is missing in the data, the app crashes while reading the list — and because it's reading the whole list at once, one bad record takes down the entire screen instead of just skipping that row.

Technical: `json['operator'] as Map` (and the chained casts) are non-nullable while the model's fields are nullable. Fix: null-safe casts / skip malformed records.

Medium

C7 MEDIUM A failed sync can create duplicate sessions on the server

flutter · lib/features/session_sync/data/repositories/
work_session_sync.data.repository.dart:233

In plain terms: When an offline recording is synced, the app first creates the session on the server, then saves the server's ID locally. If that local save fails, the app forgets the server ID — so the next sync attempt creates the session on the server *again*. You end up with two copies of the same recording on the server, and the upload attaches to only one.

Technical: no idempotency key on `createWorkSession`; if `saveOne` throws after the POST, the local row stays `notSynced` with a null remote id and re-creates on retry. Fix: idempotency key or persist the remote id before anything that can fail.

C8 MEDIUM The sensor clock is needlessly re-synced on every recording

flutter · lib/features/measurements/data/repositories/
ble_measurements.data.repository.dart:142

In plain terms: There's an optimization meant to skip re-syncing a sensor's clock if it was synced in the last 30 minutes. A units mix-up (milliseconds treated as microseconds) makes the "time since last sync" always look enormous, so the sync always runs. It's not harmful to the data, but it adds extra Bluetooth round-trips and delay to the start of every recording, and the optimization is effectively dead.

Technical: `utcTime` (ms) is passed to `DateTime.fromMicrosecondsSinceEpoch`, so the computed age is ~1000× too large → always > 30 min. Fix: use `fromMillisecondsSinceEpoch`.

C9 MEDIUM Session list screen leaks memory and background work

flutter · lib/features/session/.../session_list_tile.widget.dart:73

In plain terms: Each row in the session list creates several internal controllers (including active sync/upload work), but when a row scrolls off screen it only cleans up one of them and leaves the rest running. On a long list this piles up — memory and open network/Bluetooth work grow the more you scroll, and a sync you started can keep running after you've navigated away.

Technical: `dispose()` only disposes the `AnimationController`; four cubits (incl. a sync subscription and cancel tokens) are never `close()`d. Fix: close all four in `dispose()` (a sibling screen already does this correctly).

C10 MEDIUM Work-cycle step boundaries are never sent to their dedicated endpoint

flutter · lib/features/work_cycle/data/sources/remote/work_cycle.remote.datasource.dart:185

In plain terms: There's a fully-built path to send precise per-step boundaries of a work-cycle to a dedicated backend endpoint — but nothing in the app actually calls it. Instead, step boundaries are only saved as generic session events, and the task each one belongs to is left blank. If the backend reporting expects that dedicated data, it's silently lost. This is a half-finished migration.

Technical: `CreateWorkCycleEventUsecase` is registered but never injected/called; the active flow records a generic event and never sets `taskId`. Fix: wire the usecase into the cubit, or remove the dead path and set `taskId` on the events actually sent.

C11 MEDIUM Organization list fetch can crash on an unexpected response

flutter · lib/features/auth/data/sources/remote/organization.remote.datasource.dart:20

In plain terms: When fetching the list of organizations, the app assumes the server always returns a plain list. If the server returns something else (an error object, or a paginated wrapper), the app crashes in a way nobody catches, and the organization screen is stuck loading.

Technical: `response.data as List` throws a non-Dio `TypeError` that the `on DioException` catch misses and no layer handles. Fix: validate the shape before casting.

C12 MEDIUM Passwordless login can hang on an unexpected response

flutter · lib/features/auth/data/sources/remote/auth.remote.datasource.dart:53

In plain terms: The email-link ("passwordless") sign-in expects the server's reply to always contain a "message" field. If it doesn't, the app crashes uncaught and the login spinner never resolves — the user is stuck.

Technical: returns `response.data['message']` typed non-null; a missing key yields null/TypeError missed by both the Dio and repository catches. Fix: null-guard the field.

C13 MEDIUM Bluetooth scan can leak and mix results when switching modes

flutter · lib/features/bluetooth/data/sources/remote/ble.remote.datasource.dart:37

In plain terms: There are two kinds of Bluetooth scan. If the app starts one kind and then the other without stopping the first, the first scan keeps running in the background — it isn't shut down. That old scan keeps the Bluetooth radio busy and can leak its stale devices into the new scan's results.

Technical: each branch cancels only its own transport's subscription; switching without `stopScanningForBleDevices()` leaves the other alive, still feeding the newly-recreated controller. Fix: cancel both subscriptions up front.

Low

C14 LOW First heart-rate reading after a pause is mistimed

flutter · lib/features/measurements/domain/usecases/ble/
get_session_hr_readings.usecase.dart:30

In plain terms: If a heart-rate stream pauses and resumes, the first reading after the resume measures its time gap against the reading from *before* the pause — so that gap is wrongly stretched across the whole pause. It slightly skews the heart-rate recovery calculation.

Technical: the instance field `dataRef` isn't reset on restart (only `_timeStamp` is), so the first post-resume `timeDelta` spans the pause. Fix: reset `dataRef` per stream (a sibling usecase uses a local variable correctly).

C15 LOW Switching organizations offline pretends to succeed

flutter · lib/features/auth/data/repositories/auth.data.repository.dart:274

In plain terms: If a user switches their active organization while offline, the app reports success but never actually tells the server (it skips the call when there's no internet). The user believes they're now in Organization B, but the server still has them in A — so subsequent data can be filed under the wrong organization.

Technical: `setCurrentUserOrganizations` only calls the backend when online yet returns `Right(null)` either way. Fix: fail (or queue) when offline instead of reporting success.

C16 LOW A token-refresh default is inconsistent (latent trap)

flutter · lib/features/auth/domain/repositories/auth.domain.repository.dart:52

In plain terms: A helper that fetches an auth token has two conflicting default settings in two places (one says "force a refresh," the other says "don't"). No one calls it yet, so it's harmless today — but whoever uses it later could get surprising behaviour depending on how they call it. Worth fixing before it bites.

Technical: interface declares `forceRefresh = true`, implementation `= false`; Dart resolves the default from the static type at the call site. Fix: make the defaults match.

Recommended order of work

A pragmatic path to a safe production release, most urgent first.

Order	What	Why now
1	A1 + A3 — restore work_tags tenant isolation	Live cross-company data leak and tampering. Single file, single root cause. Non-negotiable before prod.
2	A2 — admin app base URL	Admin panel is completely non-functional in production. One-line fix.
3	A4, A5, A6, A7 — remaining access holes	Same class as #1 (tickets, AI report, activity re-pointing, orphan tasks). Close them together.
4	A10 + A11 — session merge	Merging is a core feature that currently fails and/or corrupts results every time.
5	A8, A9, A15–A17 — calculation accuracy	These silently produce wrong risk scores — the product's core output.
6	C1, C2, C3 — mobile data integrity	Silent data loss/corruption in what mobile records and uploads.
7	B10 then B1–B9 — mobile migration	Port condition-at-start onto dev, then fix the wire contracts, before the backend retires the old endpoints. Mobile cannot ship against the migrated backend until this is done.
8	A12–A14, A18–A28, C4–C16, B11–B15	Reliability, UX, and correctness cleanups — schedule across the next iterations.

One open question to confirm

All backend fixes were verified against branch staging (the branch that maps to the staging environment and then to production). Please confirm which branch each environment actually deploys — if any environment deploys dev instead, the backend fixes reviewed here are **not** live there, because dev is 631 commits behind and doesn't even contain the new work_library module.

Method note: findings were produced by three independent multi-agent review passes over the actual source, each followed by an adversarial verification pass whose task was to disprove the finding by re-reading the code. Only findings that survived are included. Every item cites the exact file and line so a developer can go straight to it. No production data was modified during this audit; database access, where used, was read-only.